1. **What is the computational complexity of calculating an N-gram frequency profile of a given text document with C number of words? You can use the big-Oh notation. Justify your answer. You can assume an upper limit on the word length.**

   The overall time complexity of calculating an N-gram frequency profile involves several steps: reading the file, tokenizing the text and counting word frequencies, and generating N-grams and counting their frequencies. We'll discuss each of these steps separately.

   1. Reading the file:
      This step involves going through every word in the document. If there are C words in the document, this process has a time complexity of O(C).

   2. Tokenizing the text and counting word frequencies:
      After reading the file, the program uses regular expressions to tokenize the text and creates a frequency dictionary of the tokens. This process also has a time complexity of O(C), assuming the operations of a dictionary (insertion, update, lookup) are O(1) on average.

   3. Generating n-grams and counting their frequencies:
      This is the most computationally intensive task. For each word, the program generates N-grams of sizes N and counts their frequencies. If we let N be the size of the n-grams list, and K be the maximum length of a word, then this process has a time complexity of O(CNK).

   As the generation of the n-grams and counting their frequencies is the most computationally intensive task, the overall time complexity for the generation of an N-gram frequency profile simplifies to O(CNK).

2. **What is the computational complexity of comparing two N-gram frequency profiles of length K? You can use the big-Oh notation. Justify your answer.**

   1. Generating language n-gram indices:
      This step involves creating a dictionary that maps each n-gram in the language profile to its index. The time complexity of this operation is O(K) because we're iterating over each n-gram in the profile.

   2. Calculating the distance:
      This operation involves iterating over each n-gram in the language profile and computing its difference in indexes between the document profile and language profile or assigning a penalty if the n-gram is absent in the document profile.
      The lookups of n-gram ranks in both the document profile and the language profile are done in constant time, thanks to the use of dictionaries (document_profile and language_ngram_indices). As such, each iteration involves a constant amount of work, so the total time complexity of this operation is proportional to the number of n-grams, i.e., O(K).

   Therefore, the overall time complexity of comparing two N-gram frequency profiles of length K is O(K).